**VELLORE INSTITUTE OF TECHNOLOGY**

School of Computer Science and Engineering

Fall Semester 2026–27

---

**BCSE307P · Compiler Design Lab**

Slot L49+L50 · Class Number VL2026270103362

# ICG WITH BACKPATCHING

One-Pass Intermediate Code Generation for
Control-Flow Statements using Flex and Bison

## Assessment 10 · Project Demo and Outcome

Project Completion Report

**Team 15**

| | |
|---|---|
| Prithvi Raghu | 24BCE2624 |
| Jai Narayan Jayaprakash Jayanthi | 24BCE2751 |
| Divyanshu Pal | 24BCE2856 |

Faculty: Dr. R. Naga Priyadarsini

16 October 2026

# Contents


**1 Introduction** **1**

1.1 Project overview 1

1.2 Position at Review 2 1

1.3 Scope of Review 3 1

**2 Completed Implementation** **2**

2.1 do-while and for 2

2.2 break and continue 3

2.3 Step-by-step trace 4

2.4 Symbol table and quadruple output 5

2.5 Two-pass baseline 7

**3 Project Demonstration** **8**

3.1 Demo plan 8

3.2 The hardest program 8

3.3 Completion checklist 10

**4 Results and Evaluation** **11**

4.1 Regression suite 11

4.2 Comparison of the two approaches 12

4.3 Randomised comparison 13

4.4 Problems solved 14

**5 Individual Contribution** **15**

5.1 Prithvi Raghu 15

5.2 Jai Narayan Jayaprakash Jayanthi 15

5.3 Divyanshu Pal 15

**6 Outcome: Manuscript** **16**

6.1 Contributions 16

6.2 Key results 17

6.3 Manuscript structure 17

**7 Conclusion and Future Work** **18**

**References** **18**

**Appendix A. Reproducing the Demonstration** **19**

# List of Figures


| | | |
|---|---|---|
| Figure 2.1 | do-while and for loops (tests/07_do_while.src, tests/09_for.src) | 3 |
| Figure 2.2 | continue and break in a for loop, and a break outside any loop | 4 |
| Figure 2.3 | A live --step session, paused after eight presses of Enter | 5 |
| Figure 2.4 | Symbol table and quadruple table for a bubble-sort skeleton (examples/e4_sort.src) | 6 |
| Figure 2.5 | The baseline's code before label resolution, and diffs against icg | 7 |
| Figure 3.1 | Three nested loop kinds with break and continue at different depths (examples/e3_deep.src) | 9 |
| Figure 4.1 | Full regression run: 35 of 35 checks pass | 12 |
| Figure 4.2 | Statistics from both generators on the four stress programs | 13 |
| Figure 4.3 | Randomised differential test on 500 programs | 14 |


# List of Tables


| | | |
|---|---|---|
| Table 1.1 | Review 3 scope from the project plan | 1 |
| Table 1.2 | Evaluation criteria for Project Demo and Outcome | 1 |
| Table 2.1 | Semantic actions for the loops completed at Review 3 | 2 |
| Table 2.2 | Command-line options of the finished generator | 5 |
| Table 3.1 | Demonstration sequence | 8 |
| Table 3.2 | Loop exits in Figure 3.1, checked by hand | 9 |
| Table 3.3 | Objectives and completion | 10 |
| Table 4.1 | Regression tests added for Review 3 | 11 |
| Table 4.2 | icg versus twopass | 13 |
| Table 4.3 | Problems met during implementation | 14 |
| Table 5.1 | Contribution of each team member | 15 |
| Table 6.1 | Results reported in the manuscript | 17 |
| Table 6.2 | Sections of the manuscript | 17 |

# 1 Introduction

## 1.1 Project overview

The project is a one-pass intermediate code generator for a small C-like language, written in Flex and Bison. It translates assignments, arithmetic, short-circuit boolean conditions and all the structured control flow of C into numbered three-address code. Forward jumps, whose targets do not exist when they are emitted, are resolved by backpatching (Aho et al. [1], sec. 6.7): each such jump is emitted empty, recorded in a list, and filled as soon as its target is known. A trace mode makes this normally invisible process observable, which is the project's novelty.

## 1.2 Position at Review 2

At Review 2 (16 October) the generator handled assignment, arithmetic, `if`, `if-else` with dangling-else resolution, `while`, short-circuit `&&`, `||` and `!`, constant conditions, nesting to any depth, lexical and syntax errors with line numbers, and the `--trace` mode. The grammar was conflict-free and all regression tests for that scope passed.

## 1.3 Scope of Review 3

**Table 1.1 Review 3 scope from the project plan**

| Planned item | Status | Described in |
|---|---|---|
| `do-while` and `for` loops | Complete | Section 2.1 |
| `break` and `continue` | Complete | Section 2.2 |
| Step-by-step trace | Complete | Section 2.3 |
| Symbol table and quadruple output | Complete | Section 2.4 |
| Two-pass baseline and comparison | Complete | Sections 2.5, 4.3 |
| Manuscript | Complete | Chapter 6 |

**Table 1.2 Evaluation criteria for Project Demo and Outcome**

| Criterion | Marks | Addressed in |
|---|---|---|
| 100% of project completion, demo | 3 | Chapters 2, 3 and 4 |
| Individual contribution | 3 | Chapter 5 |
| Outcome (manuscript) | 4 | Chapter 6 |

# 2 Completed Implementation

## 2.1 do-while and for

In a `do-while` loop the body comes first and the condition last, so the jump back to the start is a true exit of the condition rather than a separate `goto`. The `for` loop is the one construct whose source order differs from its execution order: the step is written before the body but runs after it. Because a one-pass generator emits code in source order, the step is emitted before the body and the pieces are connected with jumps, giving the layout init, test, step, jump to test, body, jump to step.

**Table 2.1 Semantic actions for the loops completed at Review 3**

| Production | Semantic action |
|---|---|
| `S → do W S1 while ( M B ) ;` | backpatch(S1.nextlist, M.instr); backpatch(B.truelist, W.instr)<br>S.nextlist = merge(B.falselist, breaks); continues go to M.instr |
| `S → for ( A1 ; M1 B ; M2 A2 N ) W S1` | backpatch(B.truelist, W.instr); backpatch(N.nextlist, M1.instr)<br>backpatch(S1.nextlist, M2.instr); emit(goto M2.instr)<br>S.nextlist = merge(B.falselist, breaks); continues go to M2.instr |
| `S → break ;` | emit(goto _) and add it to the innermost loop's breaks |
| `S → continue ;` | emit(goto _) and add it to the innermost loop's continues |

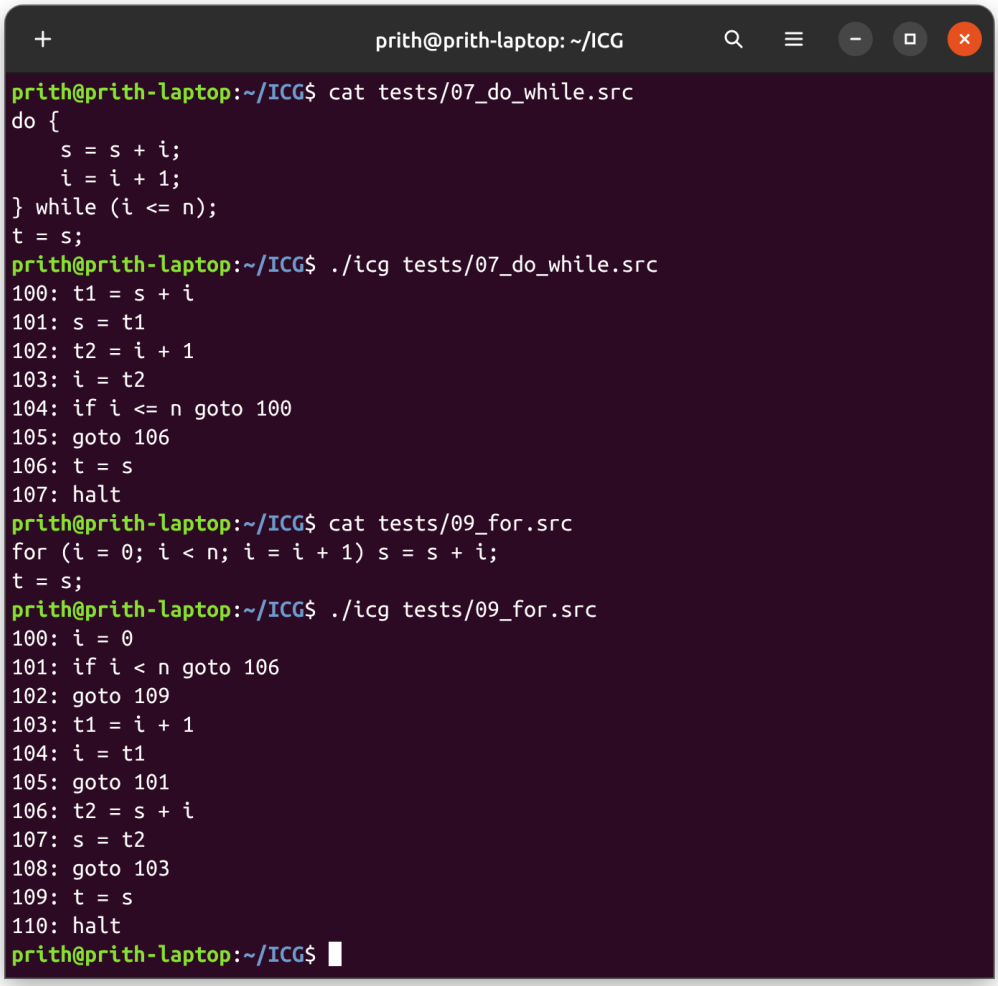



**Figure 2.1 do-while and for loops (`tests/07_do_while.src`, `tests/09_for.src`)**

In the `do-while`, quad 104 is the condition's true exit back to the body at 100. In the `for` loop, 100 is the init, 101–102 the test, 103–105 the step ending with the `N` jump back to the test, and 106–108 the body ending with the jump to the step. The test's false exit 102 leaves the loop for `t = s` at 109.

## 2.2 break and continue

The textbook scheme has no attribute for a jump out of a loop: the target of a `break` (the code after the loop) and of a `continue` (the test or the step) is unknown where the statement is parsed, and the statement may sit several levels inside `if`s. The generator keeps a stack of open loops:

- the marker `W` pushes an entry with empty `breaks` and `continues` lists as a loop body begins;
- `break` and `continue` emit `goto _` into the matching list of the innermost entry, or stop with `line N: break outside a loop` if no loop is open;

- when the loop rule is reduced, `loop_end` pops the entry, fills the continues with the loop's continue target, and returns the breaks to join the loop's `nextlist`.

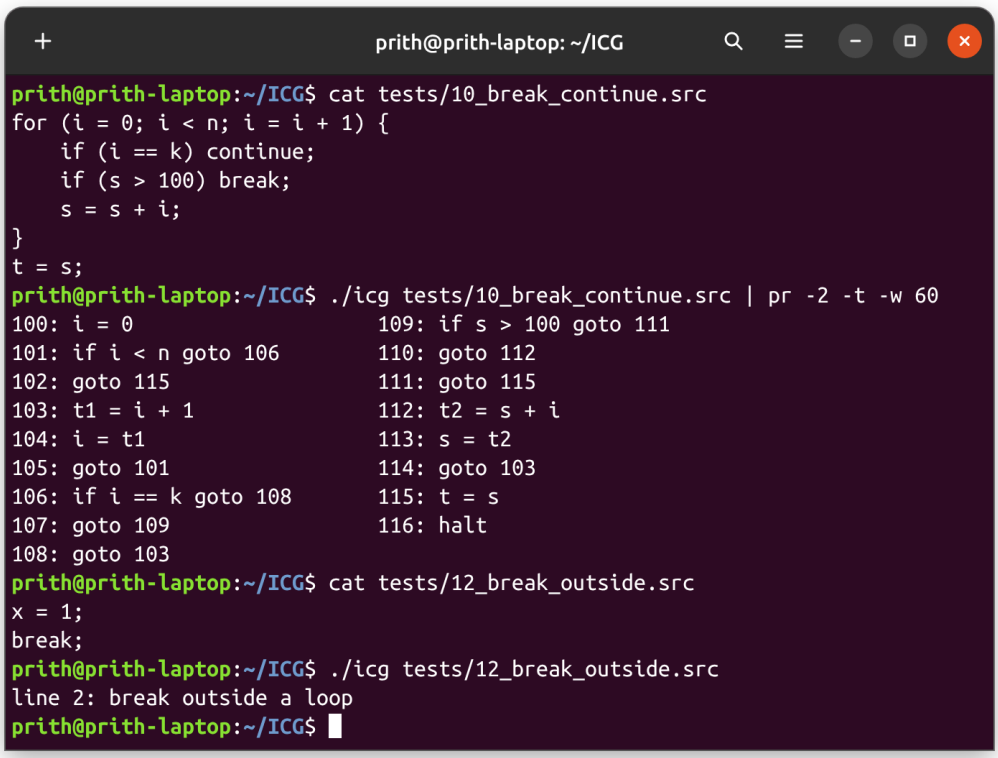



**Figure 2.2 continue and break in a for loop, and a break outside any loop**

The `continue` at 108 is filled with 103, the step of the `for`; the `break` at 111 is filled with 115, together with the loop's false exit 102. Error line numbers come from Bison location tracking, so a `break` is reported on its own line even when its semicolon is on the next one (test `19_error_line`).

## 2.3 Step-by-step trace

The Review 1 plan proposed a separate HTML viewer to replay the trace. It was delivered as `--step` instead: the generator prints the same trace as `--trace` but waits for Enter after each operation, so a class can follow one `makelist`, `merge` or `backpatch` at a time. This keeps the whole tool inside the Flex and Bison program and works in any lab terminal. Keystrokes are read from the terminal, so step mode also works when the program itself is piped in.

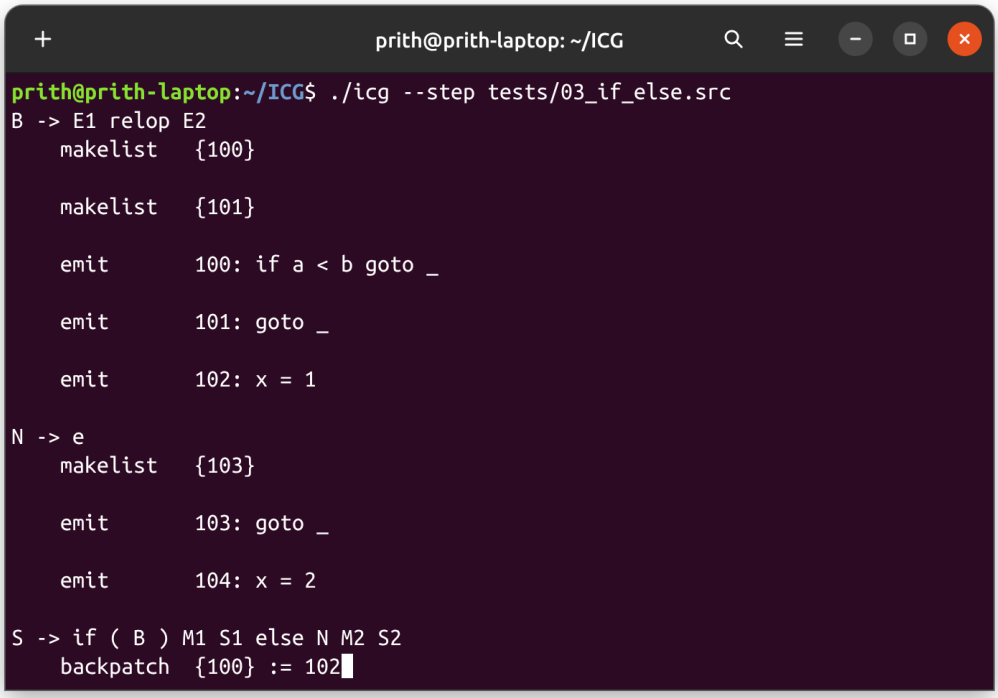



**Figure 2.3 A live `--step` session, paused after eight presses of Enter**

Each blank line in Figure 2.3 is a press of Enter. The session is paused at the moment the if-else rule fills quad 100, the condition's true exit, with 102, the start of the then-branch.

## 2.4 Symbol table and quadruple output

**Table 2.2 Command-line options of the finished generator**

| Option | Output |
|---|---|
| `--format=tac` | Numbered three-address code (default) |
| `--format=quad` | Quadruple table `op, arg1, arg2, result`; jumps use `j<`, `j==`, ... and `j` |
| `--trace`, `--step` | Every rule and list operation, then the code; `--step` pauses after each |
| `--symbols` | Each name and the line where it first appears |
| `--stats` | Quad count, peak number of unfilled jumps, patch-list entries |

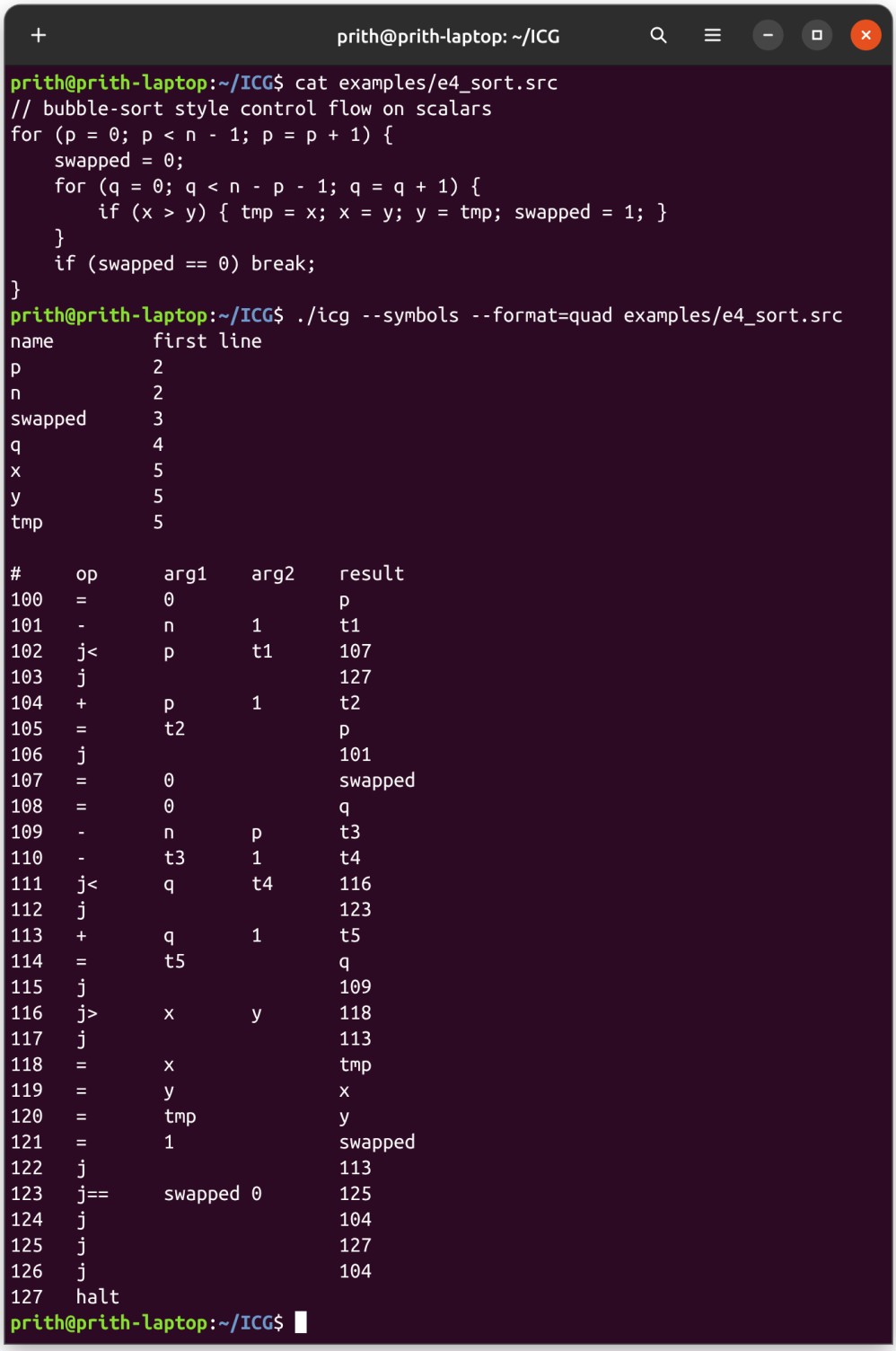



**Figure 2.4 Symbol table and quadruple table for a bubble-sort skeleton (`examples/e4_sort.src`)**

The early-exit `break` at 125 jumps to 127, joining the outer test's false exit at 103. Quad 126, the jump from the end of the outer body to its step, cannot be reached because the `if` before it either breaks or jumps to the step itself (124). Such redundant jumps are a known property of the textbook code shapes; removing them is the job of a later optimisation pass, outside this project's scope.

## 2.5 Two-pass baseline

To check the generator independently and to compare approaches, a second generator, `twopass`, was written as a separate Bison grammar (`src/twopass.y`, 501 lines) for the same language and sharing the same lexer. It follows the label-based approach of Appel [2]: pass 1 builds a syntax tree; pass 2 walks it, giving each statement the label of the code that follows and each condition its true and false labels; pass 3 replaces each label with a quad number. Its code shapes match `parser.y`, so for any program the two must print identical output; any wrong `backpatch` in `icg` would show up as a difference.

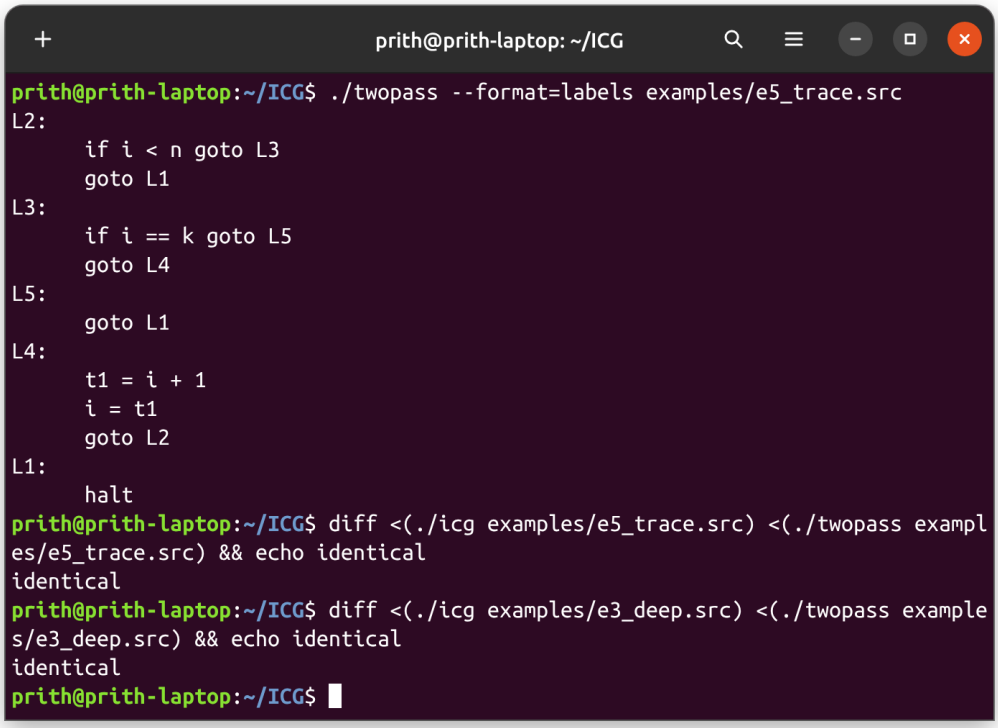



**Figure 2.5 The baseline's code before label resolution, and diffs against `icg`**

After resolution, `L2` becomes 100, `L3` 102, `L5` 104, `L4` 105 and `L1` 108, exactly the targets `icg` fills by backpatching. Both `diff` commands report no difference.

# 3 Project Demonstration

## 3.1 Demo plan

The live demonstration follows the sequence below. Every step is a command run on the finished project, and each produces the output shown in the figure listed.

**Table 3.1 Demonstration sequence**

| # | Command | What it shows | Figure |
|---|---|---|---|
| 1 | `make && make test` | Conflict-free build; all 35 checks pass | 4.1 |
| 2 | `./icg tests/07_do_while.src, 09_for` | Loop layouts | 2.1 |
| 3 | `./icg tests/10_break_continue.src` | Loop exits filled from the loop stack | 2.2 |
| 4 | `./icg examples/e3_deep.src` | Three nested loop kinds, exits at every depth | 3.1 |
| 5 | `./icg --step tests/03_if_else.src` | Backpatching one step at a time | 2.3 |
| 6 | `./twopass --format=labels ...` | Same code from the label-based baseline | 2.5 |
| 7 | `./icg --stats`, `./twopass --stats` | Memory held by each approach | 4.2 |
| 8 | `python3 tools/fuzz.py ...` | 500 random programs, both generators agree | 4.3 |

## 3.2 The hardest program

Figure 3.1 nests all three loop kinds, places a `continue` and a `break` in the innermost `for`, a second `break` in the middle `while`, and ends with a `do-while` condition using `&&` and `!`.

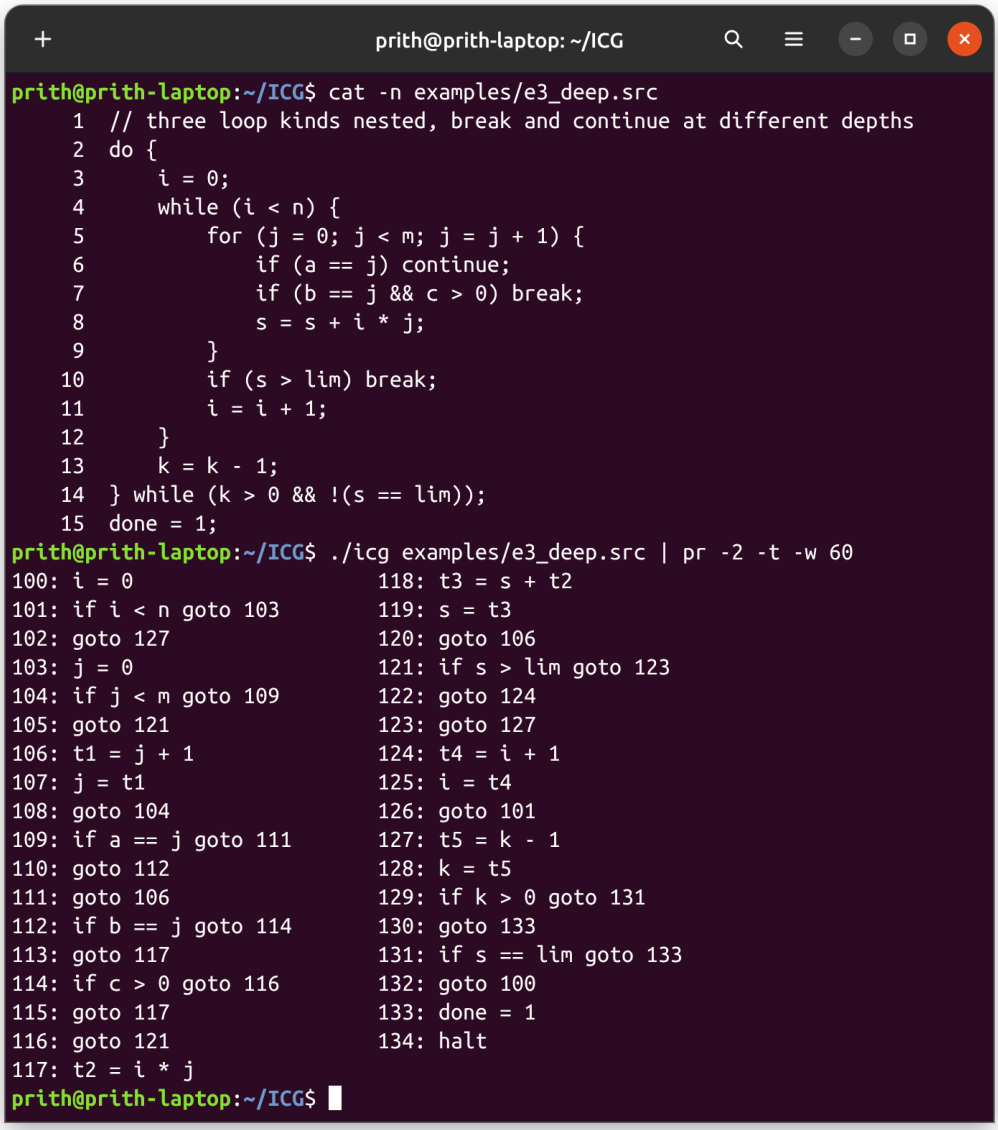



**Figure 3.1 Three nested loop kinds with break and continue at different depths (`examples/e3_deep.src`)**

**Table 3.2 Loop exits in Figure 3.1, checked by hand**

| Quad | Source | Target | Why |
|---|---|---|---|
| 111 | `continue`, line 6 | 106 | step `j = j + 1` of the innermost `for` |
| 116 | `break`, line 7 | 121 | first quad after the `for`; leaves the `for` only |
| 123 | `break`, line 10 | 127 | first quad after the `while`; leaves the `while` only |
| 105, 102 | loop tests false | 121, 127 | same targets as the two `break`s |
| 130, 131 | `do-while` condition false | 133 | `k > 0` false, or `s == lim` true under `!` |
| 132 | `do-while` condition true | 100 | back to the start of the `do` body |

Each `break` leaves exactly one loop, as in C. The loop stack, not the parse depth, decides the target: the `break` on line 7 sits inside an `if` inside a block inside the `for`, and still reaches the `for`'s entry.

## 3.3 Completion checklist

**Table 3.3 Objectives and completion**

| Objective | Evidence | Status |
|---|---|---|
| Correct three-address code for all constructs, any nesting | 19 regression tests; 5 stress programs checked by hand; 500 random programs | Met |
| Single pass, every target filled by backpatching | No tree or label pass in `icg`; test runner rejects any `goto _` | Met |
| Short-circuit booleans | Tests `05_bool`, `11_true_false`; example `e2_bool` | Met |
| Clear error reporting | Tests `08`, `12`, `13`, `19`; four error examples | Met |
| Novelty: visible backpatching | `--trace` and `--step` | Met |
| Comparison with the two-pass approach | `twopass`; identical output; statistics | Met |

# 4 Results and Evaluation

## 4.1 Regression suite

The suite has 19 programs, each with a hand-checked expected output. Sixteen are also run through `twopass` and must match exactly, giving 35 checks; any final code containing `goto _` fails regardless. All 35 pass.

**Table 4.1 Regression tests added for Review 3**

| Test | What it checks | Result |
|---|---|---|
| `07_do_while` | `do-while` loop | Pass |
| `09_for` | `for`: init, test, step, body layout | Pass |
| `10_break_continue` | `break` and `continue` in a `for` | Pass |
| `12_break_outside` | `break` outside a loop rejected | Pass |
| `14_nested_loops` | `break` in an inner loop, `continue` in the outer | Pass |
| `18_symbols` | `--symbols`, names on later lines of a statement | Pass |
| `19_error_line` | `break` and its `;` on different lines | Pass |

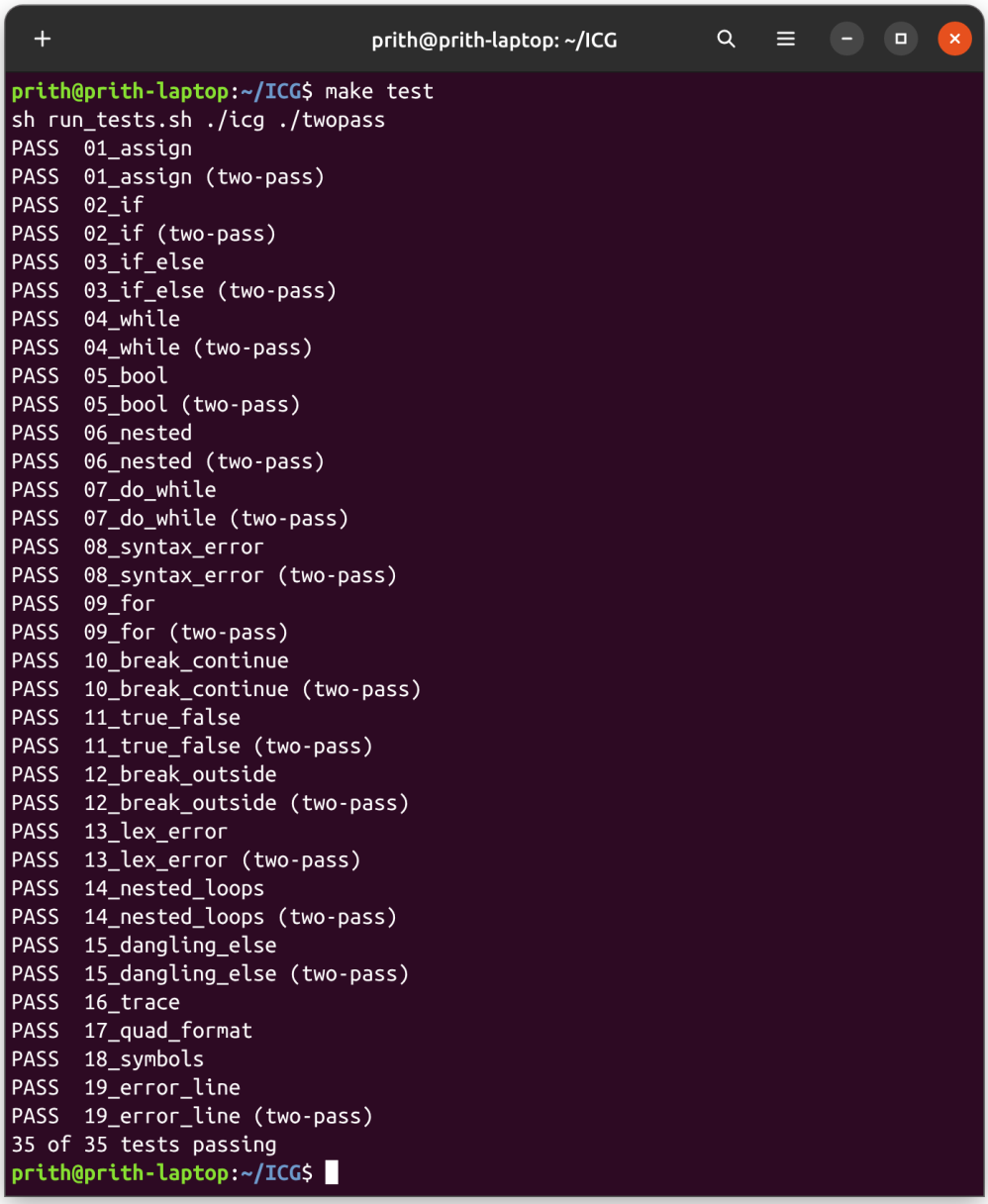



**Figure 4.1 Full regression run: 35 of 35 checks pass**

## 4.2 Comparison of the two approaches

For `icg` the key figure is the peak number of unfilled jumps, the most jumps waiting for a target at any moment: everything the one-pass generator holds beyond the code itself. For `twopass` the corresponding cost is the whole syntax tree, then the label table.

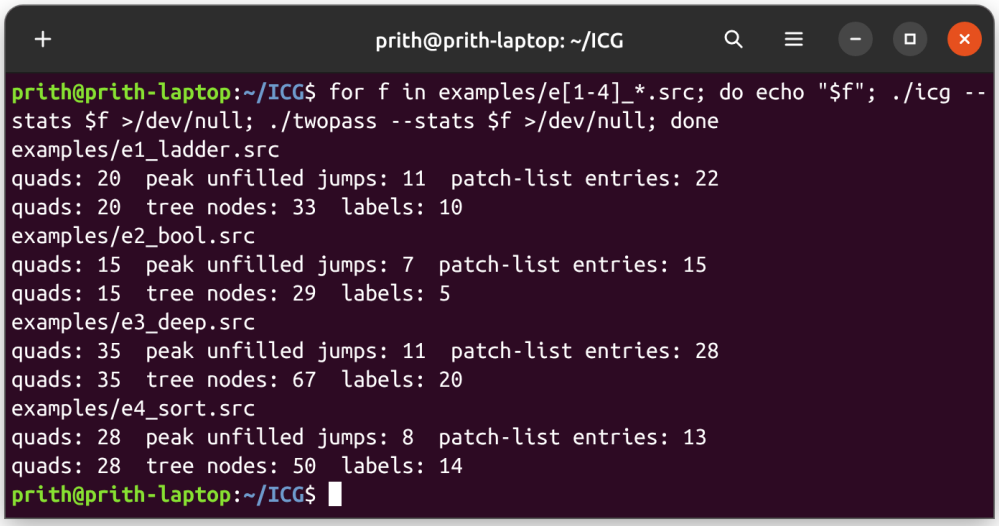



**Figure 4.2 Statistics from both generators on the four stress programs**

**Table 4.2 icg versus twopass**

| Program | Quads | Peak unfilled jumps | Patch-list entries | Tree nodes | Labels |
|---|---|---|---|---|---|
| `06_nested` | 16 | 7 | 10 | 26 | 9 |
| `10_break_continue` | 17 | 8 | 11 | 29 | 9 |
| `14_nested_loops` | 18 | 7 | 12 | 32 | 11 |
| `e1_ladder` | 20 | 11 | 22 | 33 | 10 |
| `e2_bool` | 15 | 7 | 15 | 29 | 5 |
| `e3_deep` | 35 | 11 | 28 | 67 | 20 |
| `e4_sort` | 28 | 8 | 13 | 50 | 14 |

Both generators always produce the same number of quads. On the 35-quad program the backpatching generator never has more than 11 jumps pending, while the two-pass generator builds a 67-node tree before emitting anything. The tree grows with every expression and statement; pending jumps grow only with the number of control-flow exits open at the same time. An else-if ladder is the least favourable case for backpatching, since every branch's exit waits for the end of the ladder (11 pending out of 20 quads). The two-pass approach is easier to reason about, but it needs the whole program in memory and an extra pass; backpatching delivers the same code in one pass.

## 4.3 Randomised comparison

A generator script produced random programs using every construct, nested up to four levels, with `break` and `continue` only inside loops. Each was compiled by both programs and three conditions were checked: identical output and exit status from `icg` and `twopass`; no `goto _` in the final code; and the same code with and without `--trace`.

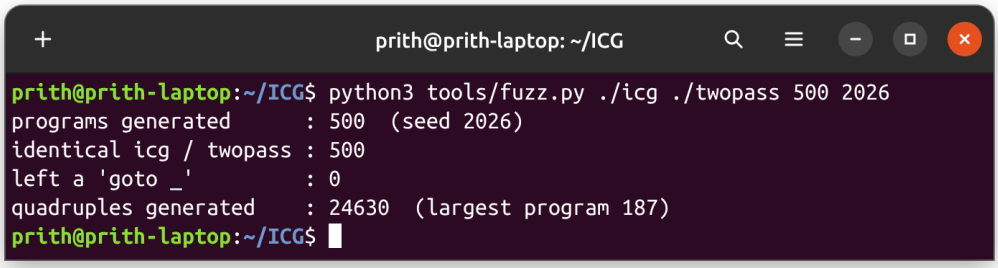



**Figure 4.3 Randomised differential test on 500 programs**

All 500 programs passed all three checks, producing 24,630 quads in total; the largest program had 187. Because the two generators were written independently, one with patch lists and one with labels, this agreement is strong evidence that every semantic action fills each jump correctly.

## 4.4 Problems solved

**Table 4.3 Problems met during implementation**

| Problem | Resolution |
|---|---|
| Reduce/reduce conflict with `N` before `else` | `N` moved after `else`, turning it into a shift/reduce choice that precedence resolves; generated code unchanged |
| No textbook attribute for `break` / `continue` | Loop stack opened by `W`; change confined to loop rules and the two jump statements |
| `for` step written before the body | Step emitted in source order and linked with an `N` jump and a backward `goto` |
| `break` error reported one line late when `;` is on the next line | `%locations`; the line of the `break` token itself is used |
| Bison precedence warnings | `%precedence` where associativity was unused; parse tables byte-identical, build warning-free |

# 5 Individual Contribution

The work was divided by component, so that each member owned a distinct part of the system.

**Table 5.1 Contribution of each team member**

| Member | Responsibilities | Main files |
|---|---|---|
| **Prithvi Raghu** 24BCE2624 | Grammar design; marker non-terminals `M`, `N`, `W`; backpatching core (quad table, `makelist`, `merge`, `backpatch`) and all semantic actions; dangling-else resolution; trace and step modes | `src/parser.y` |
| **Jai Narayan Jayaprakash Jayanthi** 24BCE2751 | Flex lexer; error handling for lexical, syntax and loop-exit errors, with line numbers from Bison locations; symbol table and `--symbols`; regression test suite and test runner | `src/lexer.l`, `tests/`, `run_tests.sh` |
| **Divyanshu Pal** 24BCE2856 | Two-pass baseline compiler; `--stats` for both generators and the comparison of approaches; project report and review presentations | `src/twopass.y`, `docs/` |

## 5.1 Prithvi Raghu

Designed the grammar and the translation scheme and wrote the code-generation core in `parser.y`: the quadruple table, the patch-list helpers, and the semantic action for every statement and condition, including the `for` layout and the loop rules. Resolved the dangling-else conflict by placing `N` after `else`, and implemented the trace and step modes that form the project's novelty.

## 5.2 Jai Narayan Jayaprakash Jayanthi

Wrote the Flex lexer for the full language and the error handling across the system, including readable token names in syntax errors and exact line numbers for misplaced `break` and `continue`. Built the symbol table and its output, and the regression suite: the test programs, their hand-checked expected outputs, and the runner that rejects any unfilled jump.

## 5.3 Divyanshu Pal

Built the two-pass baseline compiler as an independent check on the backpatching generator, added the statistics used to compare the two approaches, and prepared the project report and review presentations.

# 6 Outcome: Manuscript

The outcome of the project is a manuscript describing the generator, its novelty and its evaluation. The full manuscript is the project report (`docs/final_report.pdf`, 17 pages), accompanied by the final presentation (`docs/final_review.pptx`) and the source code. This chapter presents it in condensed paper form.

### One-Pass Intermediate Code Generation with Observable Backpatching in Flex and Bison

*Prithvi Raghu, Jai Narayan Jayaprakash Jayanthi, Divyanshu Pal*
*School of Computer Science and Engineering, Vellore Institute of Technology*

**Abstract.** A compiler that emits code while it parses meets jumps whose targets do not yet exist. Backpatching resolves them by emitting each such jump with an empty target, recording it in a list, and filling the list once the target is known. We present a one-pass intermediate code generator, written entirely as a Flex lexer and a Bison grammar, that translates a C-like language with assignments, arithmetic, short-circuit boolean conditions, `if`, `if-else`, `while`, `do-while`, `for`, `break` and `continue` into three-address code by backpatching. Loop exits, which the textbook scheme does not cover, are handled by a loop stack opened by an empty marker rule. To make the process observable, the generator traces every `makelist`, `merge` and `backpatch` operation and can replay them one step at a time. An independently written two-pass generator based on symbolic labels produces identical output on all 16 shared test programs and on 500 randomly generated programs. On the test programs the backpatching generator never holds more than 11 pending jumps, against syntax trees of up to 67 nodes in the two-pass design.

**Keywords:** intermediate code generation, backpatching, three-address code, short-circuit evaluation, Flex, Bison, compiler education.

## 6.1 Contributions

1. A complete one-pass backpatching generator in Flex and Bison for structured control flow, including `break` and `continue` through a loop stack, with a conflict-free grammar.
2. A placement of the marker `N` after `else` that avoids the reduce/reduce conflict the textbook placement causes in Bison, without changing the generated code.
3. Trace and step modes that expose the patch lists while code is generated, for use in teaching.
4. A differential evaluation against an independent two-pass, label-based generator, and a measurement of what each approach holds in memory.

## 6.2 Key results

**Table 6.1 Results reported in the manuscript**

| Measure | Result |
|---|---|
| Grammar conflicts | 0 in both grammars (`%expect 0`) |
| Regression checks | 35 of 35 pass |
| Random programs, `icg` vs `twopass` | 500 of 500 identical; no unfilled jumps; trace never alters code |
| Peak pending jumps (`icg`) | At most 11, on programs of up to 35 quads |
| Tree nodes (`twopass`) | Up to 67 on the same programs |

## 6.3 Manuscript structure

**Table 6.2 Sections of the manuscript**

| # | Section | Content |
|---|---|---|
| 1 | Introduction | The forward-jump problem; objectives; scope |
| 2 | Literature survey | Six sources; comparison of three approaches; the gap |
| 3 | System design | Architecture; language; translation scheme; markers; data structures; dangling else |
| 4 | Implementation | Lexer; actions; loop exits; errors; trace mode; two-pass baseline; tooling |
| 5 | Results and testing | Test suite; worked examples; trace; comparison table |
| 6 | Conclusion and future work | `switch`, arrays and calls, type checking, peephole optimisation, graphical replay |

# 7 Conclusion and Future Work

The project is complete. The generator translates every construct in its language into correct three-address code in one pass, fills every jump by backpatching, reports errors with exact line numbers, and makes the whole process visible through its trace and step modes. Its output agrees exactly with an independent two-pass implementation on all shared tests and on 500 random programs. The final report and presentation will be uploaded to VTOP by 30 October.

Natural extensions, outside the agreed scope, are a `switch` statement, whose case exits fit directly into patch lists; arrays and procedure calls; declarations and type checking; a peephole pass to remove the redundant jumps the textbook code shapes produce; and a graphical replay of the trace for classroom use.

# References

1. A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman. *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006. Ch. 6, sec. 6.6–6.7.
2. A. W. Appel. *Modern Compiler Implementation in C*. Cambridge University Press, 1998. Ch. 7.
3. K. D. Cooper, L. Torczon. *Engineering a Compiler*, 2nd ed. Morgan Kaufmann, 2011. Ch. 7.
4. J. Levine. *flex & bison*. O'Reilly Media, 2009.
5. C. Lattner, V. Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation." *Proc. International Symposium on Code Generation and Optimization (CGO)*, 2004, pp. 75–86.
6. S. H. Rodger, T. W. Finley. *JFLAP: An Interactive Formal Languages and Automata Package*. Jones and Bartlett, 2006.

# Appendix A. Reproducing the Demonstration

Requirements: gcc, Flex and Bison. From the project directory:

```
make                                          # build icg and twopass
make test                                     # 35 checks
./icg tests/07_do_while.src                   # Figure 2.1
./icg tests/09_for.src
./icg tests/10_break_continue.src             # Figure 2.2
./icg --step tests/03_if_else.src             # Figure 2.3, Enter to advance
./icg --symbols --format=quad examples/e4_sort.src   # Figure 2.4
./twopass --format=labels examples/e5_trace.src      # Figure 2.5
./icg examples/e3_deep.src                    # Figure 3.1
./icg --stats examples/e3_deep.src            # Figure 4.2
./twopass --stats examples/e3_deep.src
python3 tools/fuzz.py ./icg ./twopass 500 2026 # Figure 4.3
```